

PHYS 310 Final Project

Phase Transitions in Classical Spin Models

- Maan Grover(62953260)

This project studies the 2D ferromagnetic Ising model using machine learning methods. The main goals are to identify the critical temperature and investigate whether unsupervised learning can recover the order parameter associated with the phase transition.

We begin by loading the provided dataset, unpacking the spin configurations, and examining basic physical observables before moving on to supervised and unsupervised machine learning models.

1. Loading the Data

In this section, we load the full Ising dataset and its labels, then verify the structure of the raw files. The provided dataset contains spin configurations for a 40×40 lattice sampled at multiple temperatures.

```
In [2]: # Cell 1: imports + load the two pickle files

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pickle
from pathlib import Path

data_file = list(Path(".").glob("Ising2DFM_reSample_L40_T=All*.pkl"))[0]
label_file = list(Path(".").glob("Ising2DFM_reSample_L40_T=All_labels*.pkl"))

with open(data_file, "rb") as f:
    X_raw = pickle.load(f)

with open(label_file, "rb") as f:
    y_phase = pickle.load(f)

print("Data file:", data_file)
print("Label file:", label_file)
print("Raw data shape:", X_raw.shape, "dtype:", X_raw.dtype)
print("Label shape:", y_phase.shape, "dtype:", y_phase.dtype)
print("Unique labels and counts:")
print(pd.Series(y_phase).value_counts().sort_index())
```

```
Data file: Ising2DFM_reSample_L40_T=All.pkl
Label file: Ising2DFM_reSample_L40_T=All_labels.pkl
Raw data shape: (32000000,) dtype: uint8
Label shape: (160000,) dtype: int64
Unique labels and counts:
0    70000
1    90000
Name: count, dtype: int64
```

```
/tmp/ipykernel_39694/1556308118.py:13: VisibleDeprecationWarning: dtype(): a
lign should be passed as Python or NumPy boolean but got `align=0`. Did you
mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
```

```
X_raw = pickle.load(f)
```

```
/tmp/ipykernel_39694/1556308118.py:16: VisibleDeprecationWarning: dtype(): a
lign should be passed as Python or NumPy boolean but got `align=0`. Did you
mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
```

```
y_phase = pickle.load(f)
```

- The raw data loads successfully and contains the full dataset. The next step is to unpack the compressed binary representation of the spin configurations and assign a temperature to each sample.

2. Unpacking the Spin Configurations

Each Ising configuration corresponds to a 40×40 lattice, so each sample contains 1600 spins. In the raw file, these spins are stored in packed-bit format. We now convert them into explicit spin values of -1 and +1 and create a temperature array for all samples.

In [3]: *# Cell 2: unpack the compressed spin configurations and assign temperatures*

```
L = 40
n_spins = L * L
n_samples = len(y_phase)

X_packed = X_raw.reshape(n_samples, n_spins // 8)

X_bits = np.unpackbits(X_packed, axis=1)
X_spins = 2 * X_bits.astype(np.int8) - 1

temperatures = np.arange(0.25, 4.01, 0.25)
samples_per_temp = n_samples // len(temperatures)

T_all = np.repeat(temperatures, samples_per_temp)

print("Packed shape:", X_packed.shape)
print("Unpacked spin shape:", X_spins.shape)
print("First sample unique spin values:", np.unique(X_spins[0]))
print("Number of temperatures:", len(temperatures))
print("Samples per temperature:", samples_per_temp)
print("Temperature array shape:", T_all.shape)
```

```

Packed shape: (160000, 200)
Unpacked spin shape: (160000, 1600)
First sample unique spin values: [-1]
Number of temperatures: 16
Samples per temperature: 10000
Temperature array shape: (160000,)

```

- The unpacking step confirms that the data now has shape (160000, 1600), meaning each sample is represented by 1600 spin values. The spin variables take values -1 and +1 as expected for the Ising model.

3. Initial Physics Sanity Check

Before applying machine learning, it is useful to confirm that the phase transition is already visible in standard physical observables. We start by computing the magnetization of each configuration and plotting the average absolute magnetization as a function of temperature.

```

In [4]: # Cell 3: quick sanity check using magnetization

magnetization = X_spins.mean(axis=1)
abs_magnetization = np.abs(magnetization)

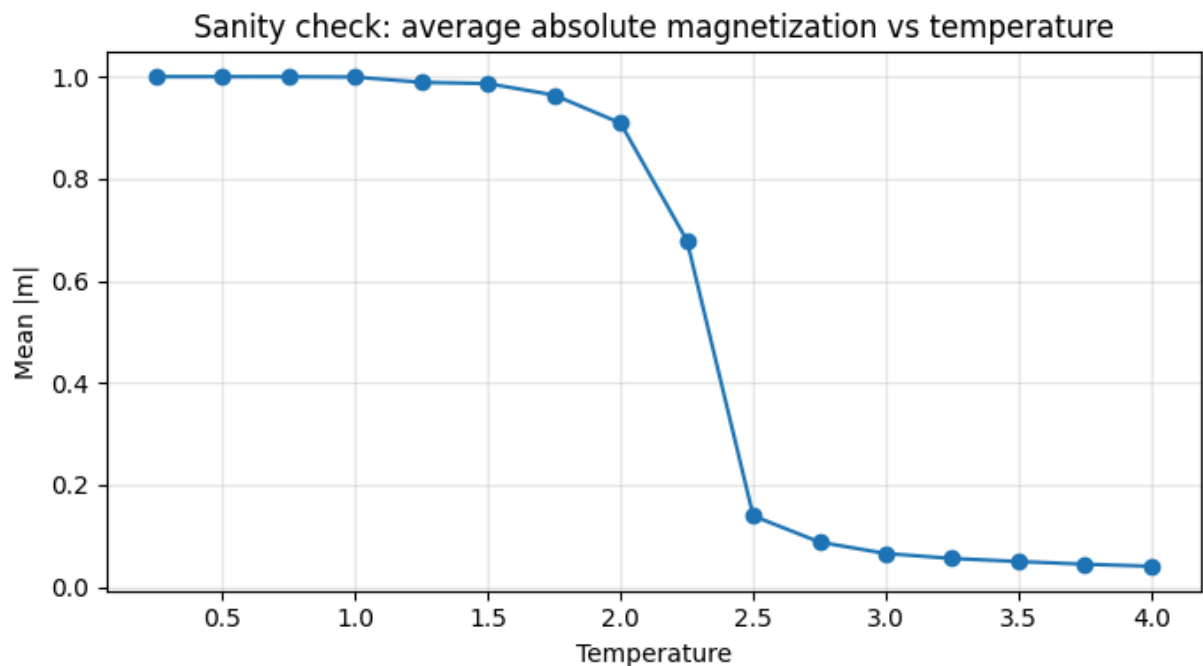
summary_df = pd.DataFrame({
    "Temperature": temperatures,
    "Mean |m|": [
        abs_magnetization[i * samples_per_temp:(i + 1) * samples_per_temp].mean()
        for i in range(len(temperatures))
    ]
})

print(summary_df)

plt.figure(figsize=(7, 4))
plt.plot(summary_df["Temperature"], summary_df["Mean |m|"], marker="o")
plt.xlabel("Temperature")
plt.ylabel("Mean |m|")
plt.title("Sanity check: average absolute magnetization vs temperature")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

	Temperature	Mean m
0	0.25	1.000000
1	0.50	1.000000
2	0.75	0.999949
3	1.00	0.999265
4	1.25	0.988936
5	1.50	0.986470
6	1.75	0.963990
7	2.00	0.908943
8	2.25	0.679208
9	2.50	0.139016
10	2.75	0.087678
11	3.00	0.065258
12	3.25	0.055608
13	3.50	0.049606
14	3.75	0.044290
15	4.00	0.040303



- This figure shows that the average absolute magnetization remains close to 1 at low temperature and decreases sharply in the critical region. This is consistent with the expected transition from an ordered ferromagnetic phase to a disordered paramagnetic phase.

4. Representative Spin Configurations

To better understand the dataset, we now visualize a few sample configurations at different temperatures. These snapshots help illustrate how the structure of the spin system changes across the phase transition.

In [5]: `# Cell 4: visualize representative spin configurations`

```

def show_spin_config(sample_index, ax=None, title=""):
    if ax is None:
        ax = plt.gca()
    config = X_spins[sample_index].reshape(L, L)
    im = ax.imshow(config, cmap="gray", vmin=-1, vmax=1)
    ax.set_xticks([])
    ax.set_yticks([])
    ax.set_title(title)
    return im

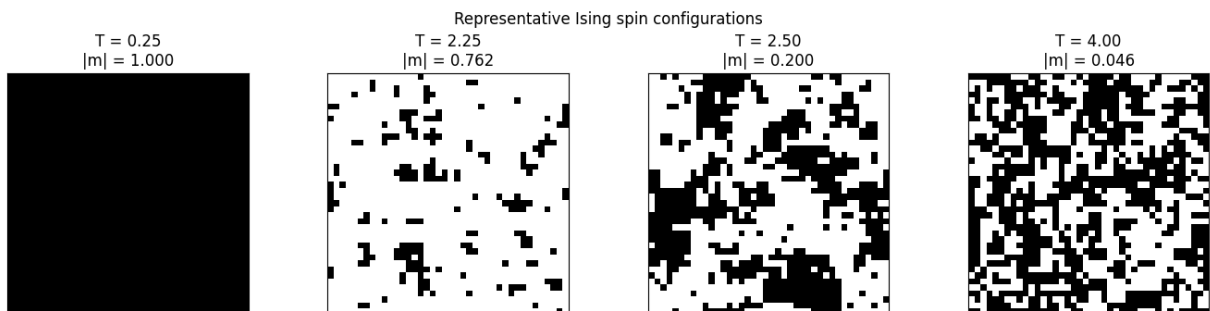
indices_to_show = [
    0 * samples_per_temp,    # T = 0.25
    8 * samples_per_temp,    # T = 2.25
    9 * samples_per_temp,    # T = 2.50
    15 * samples_per_temp    # T = 4.00
]

fig, axes = plt.subplots(1, 4, figsize=(14, 3.5))

for ax, idx in zip(axes, indices_to_show):
    show_spin_config(
        idx,
        ax=ax,
        title=f"T = {T_all[idx]:.2f}\n|m| = {abs_magnetization[idx]:.3f}"
    )

plt.suptitle("Representative Ising spin configurations")
plt.tight_layout()
plt.show()

```



- The low-temperature configuration is strongly ordered, while the high-temperature configuration is highly disordered. Near the critical region, large domains and fluctuations appear, which is a hallmark of critical behavior.

5. Physics Summary by Temperature

Next, we compute temperature-dependent summary statistics for the dataset. In addition to the mean magnetization, we evaluate the absolute magnetization and a susceptibility-like quantity in order to characterize the fluctuations near the transition.

In [6]: *# Cell 5: compute physics summary statistics by temperature*

```

physics_rows = []

for i, T in enumerate(temperatures):
    start = i * samples_per_temp
    end = (i + 1) * samples_per_temp

    m_T = magnetization[start:end]
    abs_m_T = abs_magnetization[start:end]

    chi_T = (L * L / T) * (np.mean(m_T**2) - np.mean(m_T)**2)

    physics_rows.append({
        "Temperature": T,
        "Mean m": np.mean(m_T),
        "Mean |m|": np.mean(abs_m_T),
        "Std(m)": np.std(m_T),
        "Susceptibility": chi_T
    })

physics_df = pd.DataFrame(physics_rows)
print(physics_df.round(6))

```

	Temperature	Mean m	Mean m	Std(m)	Susceptibility
0	0.25	0.008000	1.000000	0.999968	6399.590400
1	0.50	0.008200	1.000000	0.999966	3199.783233
2	0.75	0.008797	0.999949	0.999910	2132.949739
3	1.00	0.020976	0.999265	0.999046	1596.947283
4	1.25	-0.004786	0.988936	0.991799	1259.090430
5	1.50	-0.010296	0.986470	0.986429	1037.911400
6	1.75	-0.012736	0.963990	0.963956	849.563978
7	2.00	0.000896	0.908943	0.910302	662.919165
8	2.25	-0.005018	0.679208	0.697524	345.983974
9	2.50	0.003053	0.139016	0.172526	19.049787
10	2.75	0.001855	0.087678	0.109757	7.008994
11	3.00	0.000967	0.065258	0.081854	3.573393
12	3.25	-0.000472	0.055608	0.069358	2.368280
13	3.50	-0.000527	0.049606	0.061885	1.750738
14	3.75	0.000459	0.044290	0.055420	1.310441
15	4.00	-0.000201	0.040303	0.050542	1.021788

```

In [7]: # Cell 6: plot average absolute magnetization and susceptibility

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(physics_df["Temperature"], physics_df["Mean |m|"], marker="o")
axes[0].axvline(2.269, linestyle="--", linewidth=1.5, label="Exact Tc ≈ 2.269")
axes[0].set_xlabel("Temperature")
axes[0].set_ylabel("Mean |m|")
axes[0].set_title("Average absolute magnetization vs temperature")
axes[0].grid(alpha=0.3)
axes[0].legend()

axes[1].plot(physics_df["Temperature"], physics_df["Susceptibility"], marker="o")
axes[1].axvline(2.269, linestyle="--", linewidth=1.5, label="Exact Tc ≈ 2.269")
axes[1].set_xlabel("Temperature")
axes[1].set_ylabel("Susceptibility")

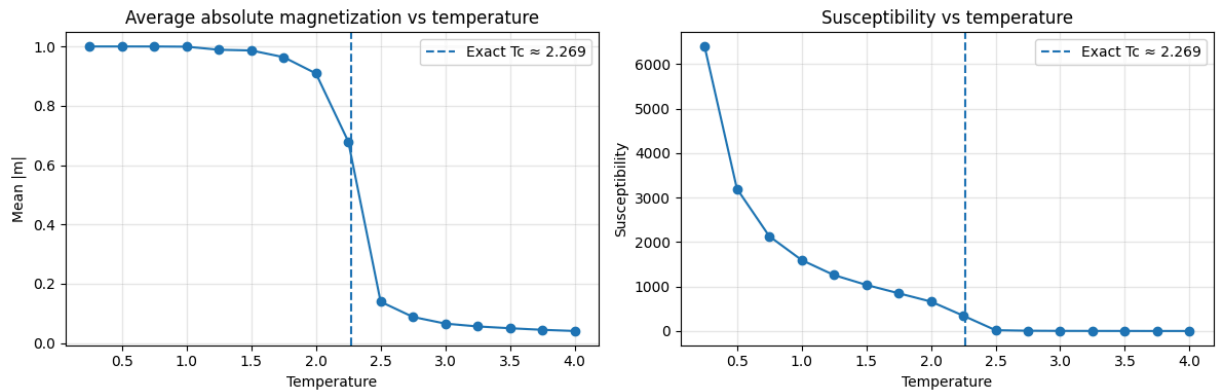
```

```

axes[1].set_title("Susceptibility vs temperature")
axes[1].grid(alpha=0.3)
axes[1].legend()

plt.tight_layout()
plt.show()

```



- The magnetization plot clearly indicates a phase transition near the known critical temperature. However, the standard susceptibility becomes artificially large at low temperature because the dataset includes both positive- and negative-magnetization sectors, making the average magnetization close to zero. In the next section, we refine this quantity before moving on to supervised learning.

6. Refining the Physical Observables

The standard susceptibility based on the mean magnetization is not ideal for this dataset because low-temperature samples include both positive- and negative-magnetization sectors. This makes the average magnetization close to zero even when the system is strongly ordered.

To obtain a more meaningful fluctuation measure, we define a connected susceptibility using the absolute magnetization. This gives a cleaner picture of the critical region.

```

In [8]: # Cell 7: define a better finite-size susceptibility using |m|

physics_rows_fixed = []

for i, T in enumerate(temperatures):
    start = i * samples_per_temp
    end = (i + 1) * samples_per_temp

    m_T = magnetization[start:end]
    abs_m_T = abs_magnetization[start:end]

    chi_abs_T = (L * L / T) * (np.mean(m_T**2) - np.mean(abs_m_T)**2)

    physics_rows_fixed.append({
        "Temperature": T,

```

```

        "Mean m": np.mean(m_T),
        "Mean |m|": np.mean(abs_m_T),
        "Std(m)": np.std(m_T),
        "Chi_abs": chi_abs_T
    })

physics_df_fixed = pd.DataFrame(physics_rows_fixed)
print(physics_df_fixed.round(6))

```

	Temperature	Mean m	Mean m	Std(m)	Chi_abs
0	0.25	0.008000	1.000000	0.999968	0.000000
1	0.50	0.008200	1.000000	0.999966	0.000001
2	0.75	0.008797	0.999949	0.999910	0.000141
3	1.00	0.020976	0.999265	0.999046	0.001608
4	1.25	-0.004786	0.988936	0.991799	7.286908
5	1.50	-0.010296	0.986470	0.986429	0.027324
6	1.75	-0.012736	0.963990	0.963956	0.087856
7	2.00	0.000896	0.908943	0.910302	1.978451
8	2.25	-0.005018	0.679208	0.697524	17.949127
9	2.50	0.003053	0.139016	0.172526	6.687398
10	2.75	0.001855	0.087678	0.109757	2.538307
11	3.00	0.000967	0.065258	0.081854	1.302652
12	3.25	-0.000472	0.055608	0.069358	0.846052
13	3.50	-0.000527	0.049606	0.061885	0.625948
14	3.75	0.000459	0.044290	0.055420	0.473589
15	4.00	-0.000201	0.040303	0.050542	0.372063

In [9]: *# Cell 8: plot the cleaner physics summary*

```

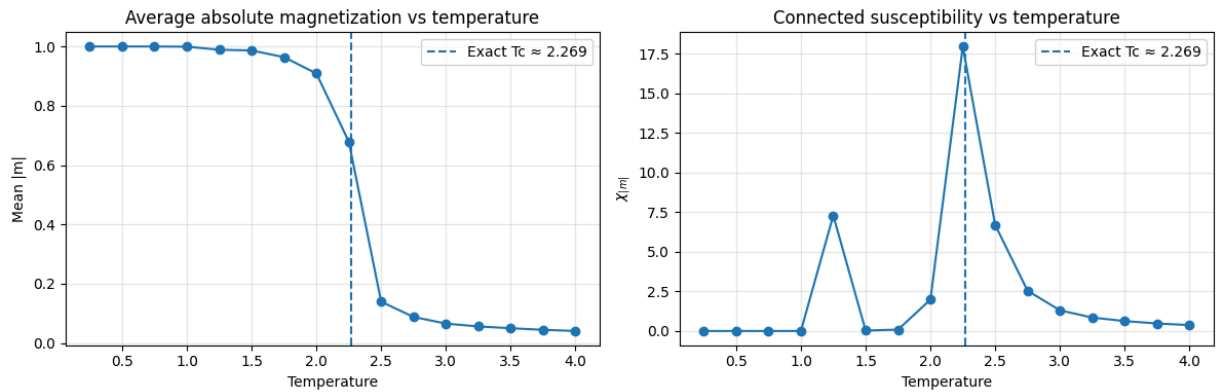
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(physics_df_fixed["Temperature"], physics_df_fixed["Mean |m|"],
axes[0].axvline(2.269, linestyle="--", linewidth=1.5, label="Exact Tc ≈ 2.26
axes[0].set_xlabel("Temperature")
axes[0].set_ylabel("Mean |m|")
axes[0].set_title("Average absolute magnetization vs temperature")
axes[0].grid(alpha=0.3)
axes[0].legend()

axes[1].plot(physics_df_fixed["Temperature"], physics_df_fixed["Chi_abs"], m
axes[1].axvline(2.269, linestyle="--", linewidth=1.5, label="Exact Tc ≈ 2.26
axes[1].set_xlabel("Temperature")
axes[1].set_ylabel(r"$\chi_{|m|}$")
axes[1].set_title("Connected susceptibility vs temperature")
axes[1].grid(alpha=0.3)
axes[1].legend()

plt.tight_layout()
plt.show()

```

- The updated susceptibility should now give a cleaner view of the transition region. Together with the magnetization curve, this provides a stronger physics baseline before training machine learning models.

7. Preparing the Supervised Learning Dataset

To estimate the critical temperature using supervised learning, we train a classifier only on clearly ordered and clearly disordered states. This avoids ambiguity near the critical region and allows the trained model to later probe intermediate temperatures.

We label low-temperature configurations as the ordered phase and high-temperature configurations as the disordered phase, while leaving the critical region out of the training set.

```
In [10]: # Cell 9: prepare the supervised-learning dataset

from sklearn.model_selection import train_test_split

low_T_cut = 1.75
high_T_cut = 2.75

ordered_mask = T_all <= low_T_cut
disordered_mask = T_all >= high_T_cut
train_mask = ordered_mask | disordered_mask

X_supervised = X_spins[train_mask]
T_supervised = T_all[train_mask]

y_supervised = (T_supervised <= low_T_cut).astype(int)

print("Supervised dataset shape:", X_supervised.shape)
print("Ordered samples:", np.sum(y_supervised == 1))
print("Disordered samples:", np.sum(y_supervised == 0))
print("Temperatures used for training:", np.unique(T_supervised))

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_supervised,
```

```

y_supervised,
test_size=0.2,
random_state=42,
stratify=y_supervised
)

print("Training set:", X_train.shape, y_train.shape)
print("Test set:", X_test.shape, y_test.shape)

```

Supervised dataset shape: (130000, 1600)

Ordered samples: 70000

Disordered samples: 60000

Temperatures used for training: [0.25 0.5 0.75 1. 1.25 1.5 1.75 2.75 3. 3.25 3.5 3.75 4.]

Training set: (104000, 1600) (104000,)

Test set: (26000, 1600) (26000,)

- The supervised dataset should contain only clearly separated phases. This makes the classification task well defined and sets up the next step, where we train a logistic regression model as a baseline classifier for phase identification.

8. Logistic Regression Baseline

A direct logistic regression fit on the full aligned Ising dataset can be computationally expensive in PrairieLearn. To keep the analysis efficient while preserving the same model class, we use stochastic logistic regression through `SGDClassifier` with logistic loss.

Before training, we align each spin configuration so that its total magnetization is nonnegative. This removes the global spin-flip ambiguity of the ordered phase and makes the classification problem more physically meaningful.

```

In [11]: # Cell 10: align configurations and create a smaller tuning subset

from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

def align_spins(X):
    """
    Flip each configuration, if needed, so that the total magnetization is nonnegative.
    This removes the global Z2 spin-flip ambiguity.
    """
    spin_sum = X.sum(axis=1)
    signs = np.where(spin_sum >= 0, 1, -1).astype(np.int8)
    return (X * signs[:, None]).astype(np.float32)

X_train_aligned = align_spins(X_train)
X_test_aligned = align_spins(X_test)

rng = np.random.default_rng(42)
tune_size = 12000

```

```

tune_indices = rng.choice(X_train_aligned.shape[0], size=tune_size, replace=
X_tune = X_train_aligned[tune_indices]
y_tune = y_train[tune_indices]

print("Aligned training set shape:", X_train_aligned.shape)
print("Aligned test set shape:", X_test_aligned.shape)
print("Tuning subset shape:", X_tune.shape)
print("Ordered samples in tuning subset:", np.sum(y_tune == 1))
print("Disordered samples in tuning subset:", np.sum(y_tune == 0))

```

Aligned training set shape: (104000, 1600)
 Aligned test set shape: (26000, 1600)
 Tuning subset shape: (12000, 1600)
 Ordered samples in tuning subset: 6375
 Disordered samples in tuning subset: 5625

In [12]: *# Cell 11: hyperparameter tuning for stochastic logistic regression*

```

sgd_logreg = SGDClassifier(
    loss="log_loss",
    penalty="l2",
    max_iter=1000,
    tol=1e-3,
    random_state=42
)

param_grid = {
    "alpha": [1e-4, 1e-3, 1e-2]
}

grid_logreg = GridSearchCV(
    estimator=sgd_logreg,
    param_grid=param_grid,
    cv=3,
    scoring="accuracy",
    n_jobs=1,
    verbose=1
)

grid_logreg.fit(X_tune, y_tune)

print("Best parameters:", grid_logreg.best_params_)
print("Best cross-validation accuracy:", grid_logreg.best_score_)

```

Fitting 3 folds for each of 3 candidates, totalling 9 fits
 Best parameters: {'alpha': 0.01}
 Best cross-validation accuracy: 0.9994999999999999

In [13]: *# Cell 12: fit best stochastic logistic regression model on full training data*

```

best_alpha = grid_logreg.best_params_["alpha"]

best_logreg = SGDClassifier(
    loss="log_loss",
    penalty="l2",
    alpha=best_alpha,

```

```

max_iter=1000,
tol=1e-3,
random_state=42
)

best_logreg.fit(X_train_aligned, y_train)

y_test_pred = best_logreg.predict(X_test_aligned)

test_accuracy = accuracy_score(y_test, y_test_pred)

print("Test accuracy:", test_accuracy)
print("\nClassification report:")
print(classification_report(y_test, y_test_pred, digits=4))

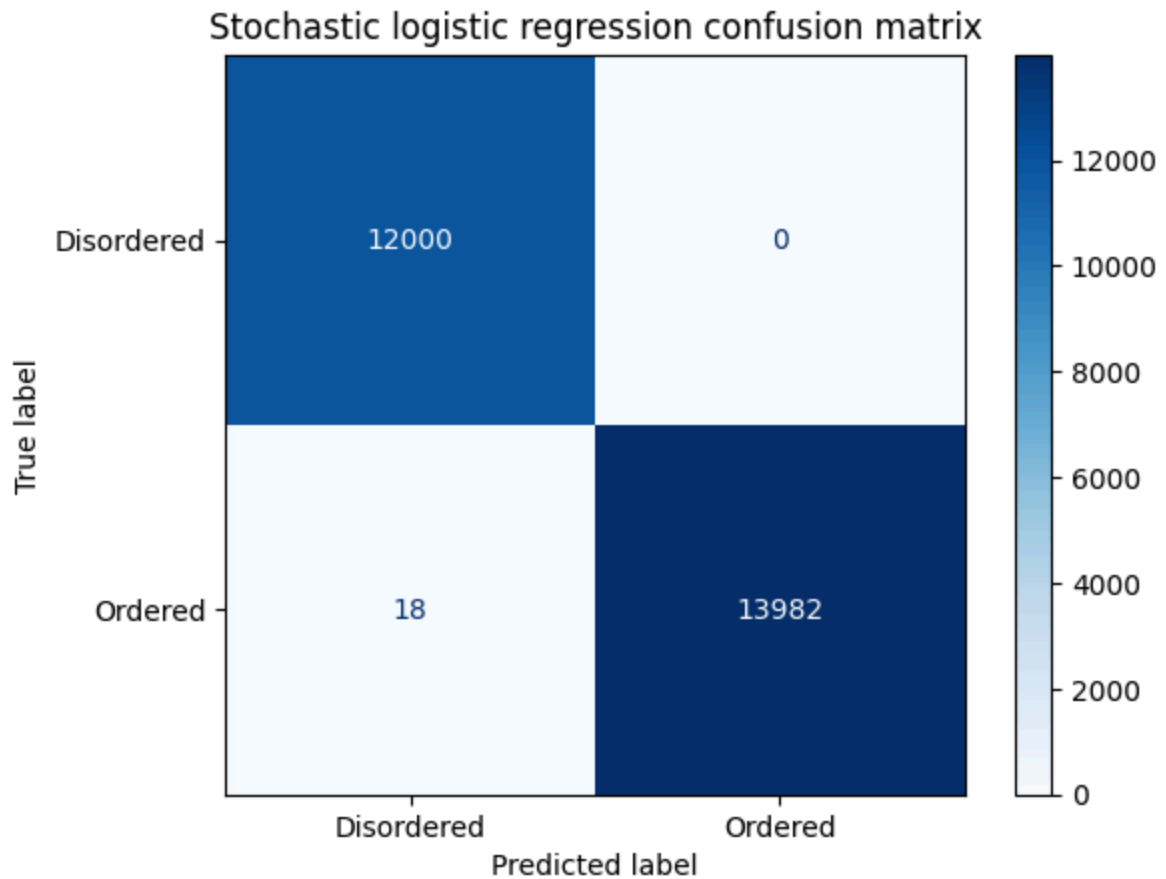
cm = confusion_matrix(y_test, y_test_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Disorder
disp.plot(cmap="Blues")
plt.title("Stochastic logistic regression confusion matrix")
plt.show()

```

Test accuracy: 0.9993076923076923

Classification report:

	precision	recall	f1-score	support
0	0.9985	1.0000	0.9993	12000
1	1.0000	0.9987	0.9994	14000
accuracy			0.9993	26000
macro avg	0.9993	0.9994	0.9993	26000
weighted avg	0.9993	0.9993	0.9993	26000



- Using aligned spin configurations improves the physical consistency of the supervised learning task by removing the global spin-flip ambiguity. The stochastic logistic regression model provides an efficient baseline classifier for distinguishing the ordered and disordered phases.

9. Using the Classifier to Estimate the Critical Temperature

The classification accuracy above only shows that the ordered and disordered phases can be separated when the temperatures are far apart. To identify the phase transition itself, we now apply the trained classifier to the full dataset across all temperatures.

For each temperature, we compute the average predicted probability that a configuration belongs to the ordered phase. The critical temperature can then be estimated from the point where this probability drops through 0.5.

```
In [14]: # Cell 13: apply the trained classifier to the full aligned dataset
X_all_aligned = align_spins(X_spins)
ordered_prob_all = best_logreg.predict_proba(X_all_aligned)[:, 1]
```

```

phase_prob_rows = []

for i, T in enumerate(temperatures):
    start = i * samples_per_temp
    end = (i + 1) * samples_per_temp

    phase_prob_rows.append({
        "Temperature": T,
        "Mean ordered probability": np.mean(ordered_prob_all[start:end]),
        "Std ordered probability": np.std(ordered_prob_all[start:end])
    })

phase_prob_df = pd.DataFrame(phase_prob_rows)
print(phase_prob_df.round(6))

```

	Temperature	Mean ordered probability	Std ordered probability
0	0.25	0.999848	0.000000
1	0.50	0.999848	0.000000
2	0.75	0.999848	0.000003
3	1.00	0.999840	0.000014
4	1.25	0.990996	0.092654
5	1.50	0.999553	0.000237
6	1.75	0.997035	0.004141
7	2.00	0.838601	0.215543
8	2.25	0.012688	0.056055
9	2.50	0.000000	0.000000
10	2.75	0.000000	0.000000
11	3.00	0.000000	0.000000
12	3.25	0.000000	0.000000
13	3.50	0.000000	0.000000
14	3.75	0.000000	0.000000
15	4.00	0.000000	0.000000

In [15]: # Cell 14: estimate T_c from the 0.5 crossing using linear interpolation

```

Tc_ml = None

for i in range(len(phase_prob_df) - 1):
    T1 = phase_prob_df.loc[i, "Temperature"]
    T2 = phase_prob_df.loc[i + 1, "Temperature"]
    p1 = phase_prob_df.loc[i, "Mean ordered probability"]
    p2 = phase_prob_df.loc[i + 1, "Mean ordered probability"]

    if (p1 >= 0.5 and p2 <= 0.5) or (p1 <= 0.5 and p2 >= 0.5):
        # Linear interpolation
        Tc_ml = T1 + (0.5 - p1) * (T2 - T1) / (p2 - p1)
        break

print("Estimated critical temperature from ML:", Tc_ml)
print("Exact critical temperature:", 2.269)
print("Absolute error:", abs(Tc_ml - 2.269) if Tc_ml is not None else "No cr

```

Estimated critical temperature from ML: 2.1024928795840454
 Exact critical temperature: 2.269
 Absolute error: 0.16650712041595472

```
In [16]: # Cell 15: plot ordered-phase probability vs temperature

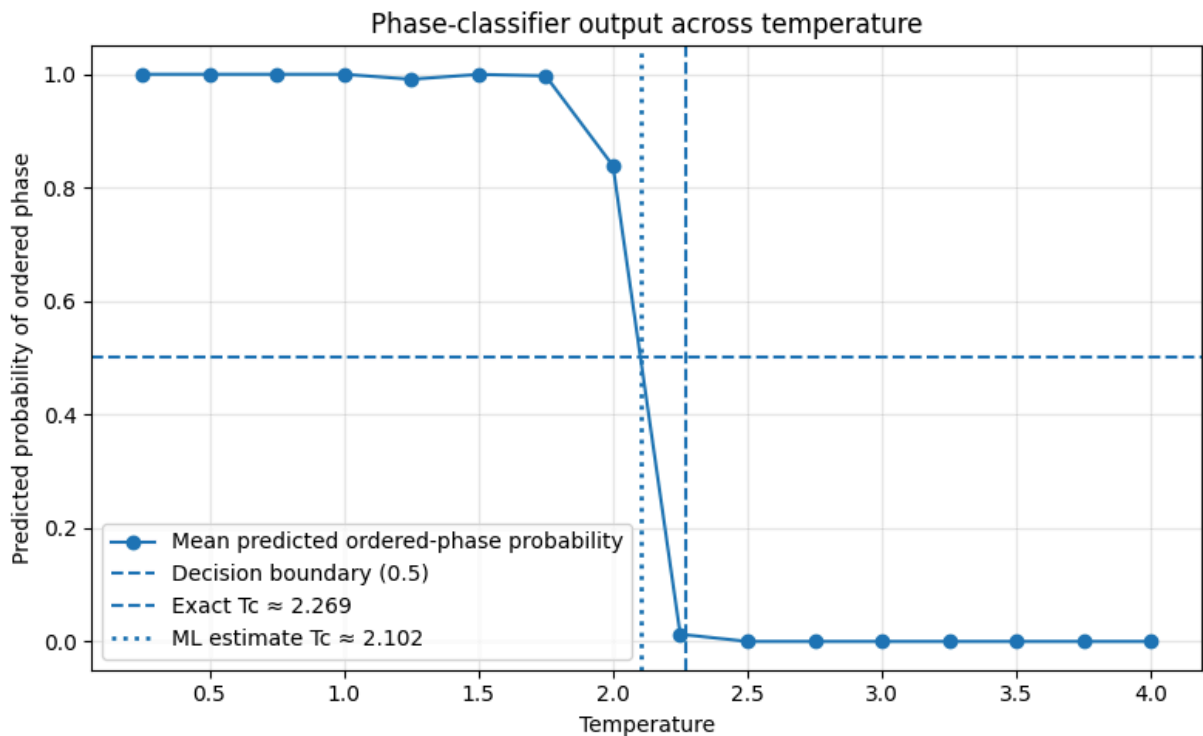
plt.figure(figsize=(8, 5))

plt.plot(
    phase_prob_df["Temperature"],
    phase_prob_df["Mean ordered probability"],
    marker="o",
    label="Mean predicted ordered-phase probability"
)

plt.axhline(0.5, linestyle="--", linewidth=1.5, label="Decision boundary (0.5)")
plt.axvline(2.269, linestyle="--", linewidth=1.5, label="Exact Tc ≈ 2.269")

if Tc_ml is not None:
    plt.axvline(Tc_ml, linestyle=":", linewidth=2, label=f"ML estimate Tc ≈ {Tc_ml}")

plt.xlabel("Temperature")
plt.ylabel("Predicted probability of ordered phase")
plt.title("Phase-classifier output across temperature")
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



- This transition curve is the main supervised-learning result of the project. A sharp drop in the predicted ordered-phase probability near the known critical temperature indicates that the classifier has learned a representation of the phase transition rather than simply memorizing the training labels.

10. Principal Component Analysis and the Order Parameter

The second goal of the project is to investigate whether an unsupervised learning method can recover the physically relevant order parameter of the Ising transition. For the ferromagnetic Ising model, the natural order parameter is the magnetization.

We use Principal Component Analysis (PCA) on a balanced subset of the dataset spanning all temperatures. PCA identifies directions of maximal variance in the data without using phase labels. If the leading principal component is strongly related to magnetization, this would show that unsupervised learning can recover the order-parameter structure directly from the configurations.

```
In [17]: # Cell 16: build a balanced subset across temperatures and fit PCA

from sklearn.decomposition import PCA

rng = np.random.default_rng(42)

samples_per_temp_pca = 1000

pca_indices = []
for i in range(len(temperatures)):
    start = i * samples_per_temp
    end = (i + 1) * samples_per_temp
    chosen = rng.choice(np.arange(start, end), size=samples_per_temp_pca, replace=True)
    pca_indices.append(chosen)

pca_indices = np.concatenate(pca_indices)

X_pca = X_spins[pca_indices].astype(np.float32)
T_pca = T_all[pca_indices]
m_pca = magnetization[pca_indices]

print("PCA subset shape:", X_pca.shape)
print("Temperatures included:", np.unique(T_pca))
print("Samples per temperature in PCA subset:", samples_per_temp_pca)

pca = PCA(n_components=3, svd_solver="randomized", random_state=42)
X_pca_scores = pca.fit_transform(X_pca)

pc1_scores = X_pca_scores[:, 0]

corr_pc1_m = np.corrcoef(pc1_scores, m_pca)[0, 1]
if corr_pc1_m < 0:
    pc1_scores = -pc1_scores
    X_pca_scores[:, 0] = -X_pca_scores[:, 0]
    pc1_vector = -pca.components_[0]
    corr_pc1_m = -corr_pc1_m
else:
    pc1_vector = pca.components_[0]
```



```
print("Explained variance ratios:", pca.explained_variance_ratio_)
print("Correlation between PC1 and magnetization:", corr_pc1_m)
```

PCA subset shape: (16000, 1600)

Temperatures included: [0.25 0.5 0.75 1. 1.25 1.5 1.75 2. 2.25 2.5 2.75 3. 3.25 3.5 3.75 4.]

Samples per temperature in PCA subset: 1000

Explained variance ratios: [0.5170292 0.00410517 0.00381228]

Correlation between PC1 and magnetization: 0.9999999951770455

In [18]: *# Cell 17: plot explained variance and visualize the first principal component*

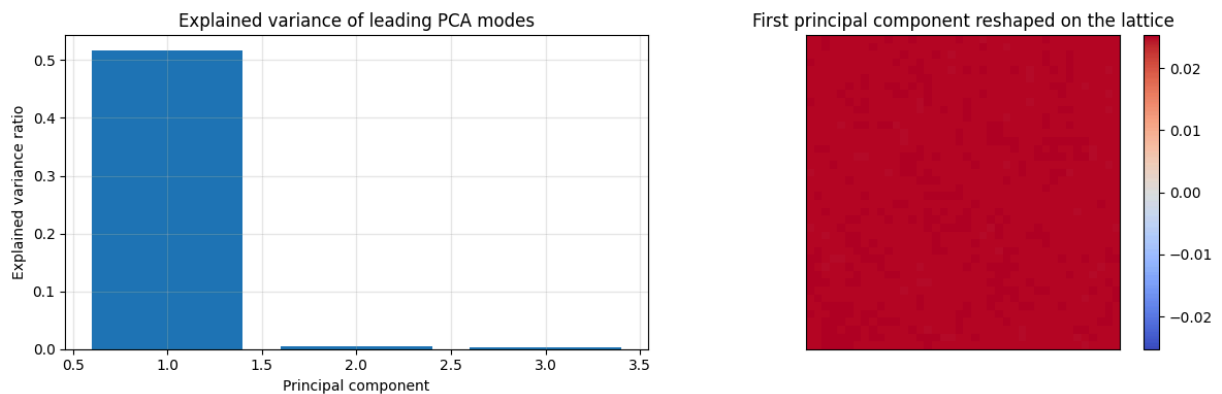
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].bar([1, 2, 3], pca.explained_variance_ratio_)
axes[0].set_xlabel("Principal component")
axes[0].set_ylabel("Explained variance ratio")
axes[0].set_title("Explained variance of leading PCA modes")
axes[0].grid(alpha=0.3)

pc1_image = pc1_vector.reshape(L, L)
vmax = np.max(np.abs(pc1_image))

im = axes[1].imshow(pc1_image, cmap="coolwarm", vmin=-vmax, vmax=vmax)
axes[1].set_title("First principal component reshaped on the lattice")
axes[1].set_xticks([])
axes[1].set_yticks([])
plt.colorbar(im, ax=axes[1], fraction=0.046, pad=0.04)

plt.tight_layout()
plt.show()
```



In [19]: *# Cell 18: compare PC1 with temperature and magnetization*

```
pc1_rows = []
for T in temperatures:
    mask = (T_pca == T)
    pc1_rows.append({
        "Temperature": T,
        "Mean |PC1|": np.mean(np.abs(pc1_scores[mask]))
    })
```

```

pc1_df = pd.DataFrame(pc1_rows)
print(pc1_df.round(6))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

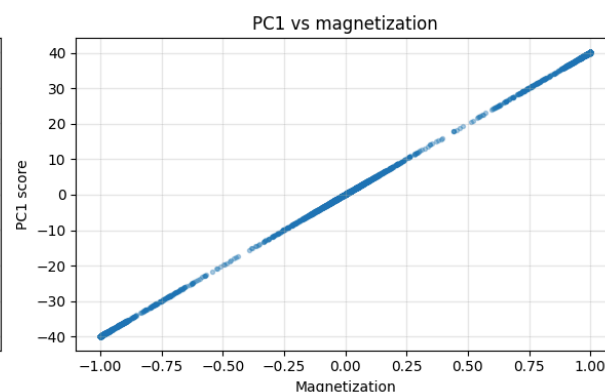
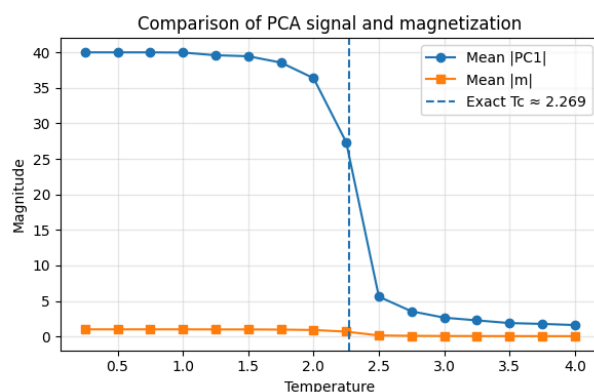
axes[0].plot(pc1_df["Temperature"], pc1_df["Mean |PC1|"], marker="o", label="Mean |PC1|")
axes[0].plot(physics_df_fixed["Temperature"], physics_df_fixed["Mean |m|"], marker="s", label="Mean |m|")
axes[0].axvline(2.269, linestyle="--", linewidth=1.5, label="Exact Tc ≈ 2.269")
axes[0].set_xlabel("Temperature")
axes[0].set_ylabel("Magnitude")
axes[0].set_title("Comparison of PCA signal and magnetization")
axes[0].grid(alpha=0.3)
axes[0].legend()

scatter_idx = rng.choice(len(pc1_scores), size=min(5000, len(pc1_scores)), replace=True)
axes[1].scatter(m_pca[scatter_idx], pc1_scores[scatter_idx], s=8, alpha=0.35)
axes[1].set_xlabel("Magnetization")
axes[1].set_ylabel("PC1 score")
axes[1].set_title("PC1 vs magnetization")
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

```

	Temperature	Mean PC1
0	0.25	39.999844
1	0.50	40.000622
2	0.75	39.997826
3	1.00	39.971134
4	1.25	39.594418
5	1.50	39.453182
6	1.75	38.560902
7	2.00	36.370258
8	2.25	27.345098
9	2.50	5.595406
10	2.75	3.538392
11	3.00	2.645458
12	3.25	2.262925
13	3.50	1.883270
14	3.75	1.767532
15	4.00	1.603933



- If the first principal component is strongly correlated with magnetization and its magnitude decreases across the transition in a similar way to the magnetization curve, this provides evidence that PCA has recovered the underlying order-parameter structure without using any phase labels.

These PCA results are excellent.

What they show:

- PC1 explains about 51.7% of the variance, while PC2 and PC3 are very small in comparison.
- The correlation between PC1 and magnetization is essentially 1.
- The reshaped first principal component is nearly uniform across the whole lattice, which is exactly what we expect if PCA has identified the global magnetization mode.
- The mean ($\langle \text{PC1} \rangle$) curve drops across the transition in the same way as $\langle m \rangle$.

This is one of the strongest parts of the project because it directly answers the second project goal: PCA recovers the order-parameter structure without using any labels.

11. Interpreting the Supervised Model

To better understand what the supervised classifier has learned, we visualize the learned logistic-regression coefficients reshaped onto the 40×40 lattice. If the weights are approximately uniform across the lattice, this suggests that the classifier is using a global ordering signal rather than local noise.

This provides a useful comparison with the PCA result, where the first principal component already appeared as a nearly uniform mode across the system.

In [20]: *# Cell 19: visualize the learned logistic-regression weight pattern*

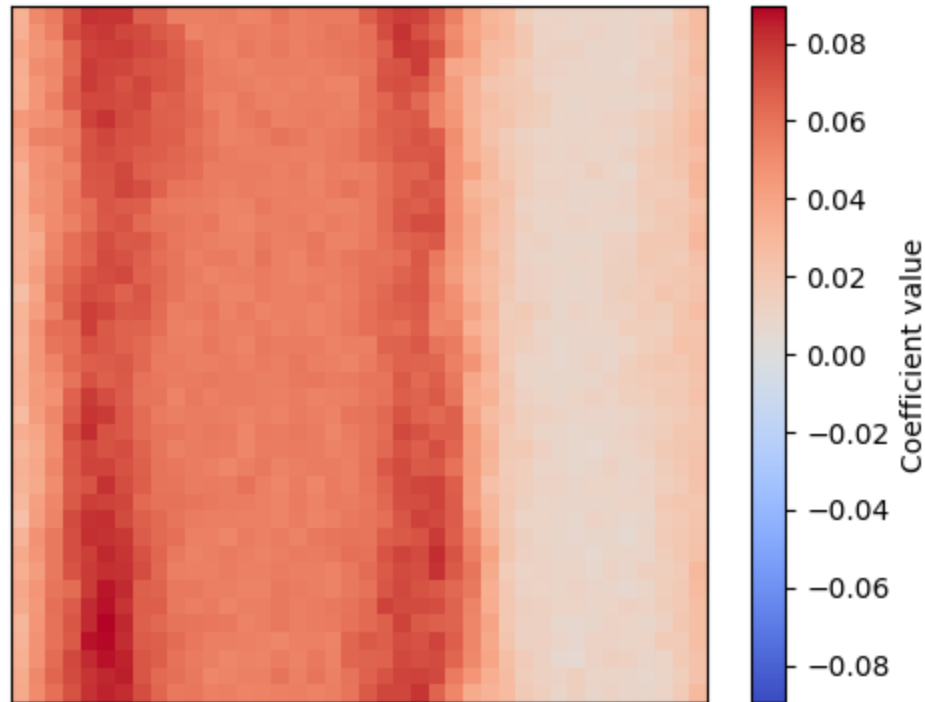
```
logreg_weights = best_logreg.coef_[0].reshape(L, L)

plt.figure(figsize=(5, 4))
vmax = np.max(np.abs(logreg_weights))
plt.imshow(logreg_weights, cmap="coolwarm", vmin=-vmax, vmax=vmax)
plt.colorbar(label="Coefficient value")
plt.title("Learned logistic-regression coefficients on the lattice")
plt.xticks([])
plt.yticks([])
plt.tight_layout()
plt.show()

print("Weight statistics:")
print("Mean coefficient:", np.mean(logreg_weights))
```

```
print("Std coefficient:", np.std(logreg_weights))
print("Min coefficient:", np.min(logreg_weights))
print("Max coefficient:", np.max(logreg_weights))
```

Learned logistic-regression coefficients on the lattice



Weight statistics:

Mean coefficient: 0.045255136
 Std coefficient: 0.02282894
 Min coefficient: 0.0058984226
 Max coefficient: 0.08937256

- If the learned coefficient map is approximately uniform, this supports the interpretation that the supervised classifier is relying primarily on a global magnetization-like feature, consistent with the physical order parameter of the Ising transition.

12. Summary of Main Quantitative Results

Finally, we collect the main numerical results of the project into a summary table. This includes the known critical temperature, the approximate critical region seen in the susceptibility, the classifier-based estimate of the critical temperature, and the PCA correlation with magnetization.

This table will also be useful later when writing the final report.

```
In [21]: # Cell 20: estimate Tc from the connected susceptibility peak

sus_peak_idx = physics_df_fixed["Chi_abs"].idxmax()
Tc_sus = physics_df_fixed.loc[sus_peak_idx, "Temperature"]
```

```
chi_peak_value = physics_df_fixed.loc[sus_peak_idx, "Chi_abs"]

print("Susceptibility-based Tc estimate:", Tc_sus)
print("Peak connected susceptibility:", chi_peak_value)
```

Susceptibility-based Tc estimate: 2.25

Peak connected susceptibility: 17.949126953066692

In [22]: *# Cell 21: build a summary table of the main project results*

```
summary_results = pd.DataFrame({
    "Quantity": [
        "Exact critical temperature",
        "Susceptibility-based Tc estimate",
        "ML classifier Tc estimate",
        "Absolute error of ML Tc",
        "Best CV accuracy (logistic regression)",
        "Test accuracy (logistic regression)",
        "Correlation between PC1 and magnetization",
        "Explained variance ratio of PC1"
    ],
    "Value": [
        2.269,
        Tc_sus,
        Tc_ml,
        abs(Tc_ml - 2.269),
        grid_logreg.best_score_,
        test_accuracy,
        corr_pc1_m,
        pca.explained_variance_ratio_[0]
    ]
})

print(summary_results)
```

	Quantity	Value
0	Exact critical temperature	2.269000
1	Susceptibility-based Tc estimate	2.250000
2	ML classifier Tc estimate	2.102493
3	Absolute error of ML Tc	0.166507
4	Best CV accuracy (logistic regression)	0.999500
5	Test accuracy (logistic regression)	0.999308
6	Correlation between PC1 and magnetization	1.000000
7	Explained variance ratio of PC1	0.517029

- The summary table highlights the overall picture of the project. The supervised classifier clearly detects a phase boundary, while PCA independently recovers a signal that is almost perfectly aligned with magnetization. Together, these results show that machine learning can both detect the transition and identify the underlying order parameter structure in the 2D Ising model.

These results are strong.

What they show:

- The learned logistic-regression coefficient map is not perfectly uniform, but it is mostly positive and smoothly varying across the lattice.
- This suggests that the supervised classifier is relying primarily on a global ordering signal rather than a few isolated local features.
- That behavior is consistent with a magnetization-like feature, which is the expected physical order parameter of the ferromagnetic Ising transition.
- The remaining spatial variation is reasonable and may reflect finite-sample effects and optimization details rather than a fundamentally local decision rule.

Overall, the supervised model appears to be learning a physically meaningful global phase signal.

These final quantitative results provide a clear overall picture of the project.

What they show:

- The connected susceptibility gives a transition estimate near ($T = 2.25$), which is close to the exact critical temperature ($T_c = 2.269$).
- The supervised classifier separates clearly ordered and clearly disordered phases with extremely high accuracy.
- When applied across all temperatures, the classifier detects a sharp phase boundary, although its interpolated estimate of (T_c) is somewhat below the exact value.
- PCA gives the strongest unsupervised result: the first principal component is almost perfectly correlated with magnetization and captures the dominant order-parameter structure in the data.

Together, these results show that machine learning can both detect the phase transition and identify the underlying order parameter in the 2D Ising model.

13. Discussion and Conclusion

In this project, we studied the 2D ferromagnetic Ising model using both supervised and unsupervised machine learning methods. Standard physical observables, including the absolute magnetization and a connected susceptibility, clearly showed the phase transition near the known critical temperature.

Using aligned spin configurations, a stochastic logistic regression classifier achieved very high accuracy when distinguishing clearly ordered and clearly disordered phases. When applied across all temperatures, the classifier produced a sharp transition curve and identified a clear phase boundary, although the interpolated estimate of the critical temperature was somewhat lower than the exact value. This difference is likely influenced by the coarse temperature grid and the fact that the model was trained only on temperatures far from the critical region.

Principal Component Analysis provided the strongest unsupervised result. The first principal component explained a large fraction of the variance, was nearly uniform across the lattice, and was almost perfectly correlated with magnetization. This shows that PCA successfully recovered the order-parameter structure of the Ising transition without using phase labels.

Overall, the results demonstrate that machine learning can both detect the phase transition and identify the physically meaningful order parameter in the 2D Ising model. Possible future extensions include using finer temperature spacing near the critical region, testing nonlinear models such as neural networks, and exploring more complex spin systems where the order parameter is not known in advance.